

Portfolio Master Spec

Five flagship projects at the intersection of quantitative finance, data science, and enterprise AI
Suite: Atlas • FedLens • BalanceLab • VolLab • RegimeOS

Design brief

Audience: Quant/Risk/Trading Managing Directors and internal Enterprise AI / Strategy teams. Target identity: enterprise AI and financial analytics builder with real quant depth, strong systems instincts, and the ability to explain complex models to senior non-technical stakeholders.

Portfolio archetype	Five separate repos with shared engineering standards plus a polished master landing repo
Optimization target	Technically impressive, medium-depth builds with real substance and premium presentation
Deployment stance	Low-cost first, open-source first, AWS/Azure signaling without enterprise spend
Math emphasis	Derivatives, stochastic calculus, practical risk math, and moderate formula visibility

1. Executive Summary

This portfolio is designed as a coherent platform suite rather than five disconnected demos. Each project proves a different capability layer: macro data systems, policy-language intelligence, institutional risk analytics, derivatives modeling, and decision-support orchestration.

The guiding thesis is that modern finance teams need builders who can combine rigorous quantitative methods, production-grade software, and agentic AI workflows inside controlled, explainable systems.

Three-minute takeaway

This person can build AI-enabled financial systems, understand the mathematics underneath them, and translate complex outputs into tools senior decision-makers can actually use.

Portfolio suite overview

Project	Primary role	Top signals	AI intensity	Math intensity
Atlas	Macro intelligence platform	Data engineering, APIs, SQL, cloud, dashboards	Medium	Medium
FedLens	Fed NLP and market-reaction copilot	LLMs, NLP, event studies, HITL review	High	Medium
BalanceLab	ALM and scenario analytics platform	Risk modeling, internal tooling, testing, explainability	High	Med-High
Vollab	Derivatives and stochastic risk lab	Quant math, pricing, numerical methods, performance	Low-Med	High
RegimeOS	Regime and decision-support orchestration	Bayesian reasoning, agents, portfolio/risk synthesis	High	Med-High

Shared build doctrine

- Every repo has two faces: a product-grade interface for decision-makers and a research-grade surface for technical review.
- Open-source first: Python, TypeScript, FastAPI, React, DuckDB/PostgreSQL, PyTorch, Hugging Face, Julia, and selective C++/Rust/Go.
- Low-cost deployment: static front ends, lightweight containers, DuckDB for embedded analytics, selective managed services only where they materially improve the demo.
- Heavy engineering standards: tests, CI, linting, typed interfaces, API contracts, data validation, benchmark harnesses, and reproducible local setup.
- Agentic AI is real, not decorative: tool-using workflows, structured outputs, human approval checkpoints, and evaluation/reporting.

2. Shared Architecture and Standards

2.1 Core technology doctrine

The suite should be intentionally polyglot, but never noisy. Each language exists for a specific job.

Language	Role in the suite	Where it appears most naturally
Python	Primary analytics, ML, APIs, orchestration, testing	All five projects
SQL	Feature layers, warehousing, analytical querying	Atlas, FedLens, BalanceLab, RegimeOS
TypeScript	Premium front end and design-system consistency	All five projects
Java	Enterprise service credibility and rules/workflow layers	FedLens, BalanceLab
Julia	High-math research modules and quant experimentation	Vollab, RegimeOS
C++	Performance-critical pricing kernels	Vollab, BalanceLab
Rust	Safe high-performance CLI/services	Vollab, FedLens or RegimeOS
Go	Lightweight orchestration and service tooling	RegimeOS, Atlas ops
R	Validation, econometrics appendix, benchmarking	Atlas, RegimeOS appendix
Perl/bash	Legacy-style parsing, build automation, scripting glue	Atlas, BalanceLab, devops tasks

2.2 Shared platform stack

- Frontend: React + TypeScript for polished interfaces, with Streamlit as a research/demo companion where speed matters.
- Backend/API: FastAPI as the default service layer; Java services where workflow/rules signaling is useful; Go/Rust only when they materially improve the system story.
- Data: DuckDB for local analytics and embedded OLAP, PostgreSQL for application state and audit tables, parquet files for intermediate data products.
- Modeling: pandas, polars, numpy, scikit-learn, statsmodels, xgboost/lightgbm, PyTorch, Hugging Face, probabilistic modeling where uncertainty matters.
- MLOps/DevOps: Docker, GitHub Actions, unit/integration tests, API contract tests, benchmark jobs, containerized local dev, reproducible make/bash workflows.
- Cloud posture: AWS for storage/compute signaling, Azure for enterprise-AI flavor and internal-app realism, with a local-first path for every repo.

2.3 Product and UX standard

Component	Standard
Executive mode	Short narrative cards, top KPIs, scenario summaries, one-click export, minimal jargon
Analyst mode	Deeper assumptions, model settings, drilldowns, residual/error views, scenario definitions
Copilot mode	Structured question-answer workflows, citations to calculations/data slices, approval actions
Research mode	Notebook links, derivation notes, benchmark results, assumptions and limitations
Design standard	Institutional-finance backbone with modern enterprise-AI polish; premium Plotly and consistent iconography

2.4 Shared engineering standard

- Every repo should expose a CLI, a local development flow, a test suite, sample data, and a README that explains both the business case and technical architecture.
- Every API should return structured JSON objects and publish a schema or pydantic model contract.
- Every model-bearing repo should include regression tests against known outputs and a lightweight evaluation report.
- Every AI repo should log prompts, tools used, outputs, approval state, and evaluation metrics for auditability.
- Every repo should support a no-cloud local run path so the portfolio stays cheap and easy to demo.

2.5 Master landing repo

Element	Spec
Purpose	Single front door that explains the suite, links each repo, and tells one coherent story
Frontend	Static React/Vite site or GitHub Pages-compatible build
Pages	Home, Portfolio thesis, Projects grid, Architecture, Case studies, Resume/About
Assets	Architecture diagrams, screenshots, GIF demos, benchmark snippets, repo badges
Integration	Shared visual identity, common badges, shared changelog style, issue/PR templates
Low-cost deploy	GitHub Pages or Azure Static Web Apps; static assets only

3.1 Atlas

Macro intelligence platform for economic state tracking, feature engineering, and executive-quality monitoring.

Business case

A strategy or treasury team needs one trustworthy macro backbone: pull data from many sources, standardize frequencies, build thematic indicators, and expose a clean API/UI for monitoring the economic regime.

Field	Spec
Primary users	Macro strategy, treasury, enterprise AI, risk leadership, and portfolio-oriented stakeholders who want a common macro data layer.
Hard-skill signals	Data engineering, API design, SQL, time-series analytics, cloud-lite ETL, dashboarding, CLI design, and documentation discipline.
Stack	Python, SQL, TypeScript, polars/pandas, DuckDB, PostgreSQL, FastAPI, React, Plotly, Streamlit, bash, Perl, optional R appendix.
Low-cost deploy	Low-cost path: DuckDB plus local parquet by default, React frontend on GitHub Pages or Azure Static Web Apps, FastAPI on a small container or local demo. Optional AWS S3 for static data snapshots.
Launch shape	MVP: Ingest 20-40 core series, build themed feature blocks, expose a macro dashboard, and generate a one-page narrative summary. Stretch: Add a lightweight nowcasting block, forecast error tracking, and agent-generated weekly strategy memos.

Architecture

- Ingestion layer: source-specific connectors pull FRED, Treasury, inflation expectations, labor, FX, and market-condition series.
- Normalization layer: resampling, calendar alignment, transformations, vintage handling, z-scores, rolling windows, and feature catalog creation.
- Serving layer: FastAPI endpoints expose metadata, latest values, histories, and regime snapshots; DuckDB handles local analytical queries.
- Presentation layer: React executive dashboard plus Streamlit research notebook launcher.

Interfaces

- CLI: atlas pull, atlas build-features, atlas snapshot, atlas export-report.
- API endpoints: /series, /feature-groups, /regime-snapshot, /chart-config, /health.
- Storage: parquet bronze/silver/gold pattern with a PostgreSQL metadata catalog.

Modeling and mathematics

- Standardization and signal math should be explicit, but not overdone. Example transformations include $z_t = (x_t - \mu_t) / \sigma_t$ and rolling annualized change measures.
- Optional factor view: use PCA to summarize inflation, labor, and financial-conditions blocks into lower-dimensional latent signals.

AI and workflow design

- Atlas does not need the heaviest agent layer, but it benefits from a macro-briefing copilot that can answer what changed this week, which features moved most, and how that compares with recent history.
- Human review is lightweight: generated summaries must cite series and computed features.

Testing and governance

- Connector tests for source parsing and schema stability.
- Feature regression tests for known macro calculations.

- API smoke tests and dashboard snapshot tests.
- Data quality checks for missing values, date gaps, and frequency mismatches.

Suggested repo skeleton

frontend/, api/, src/atlas_ingest/, src/atlas_features/, data_contracts/, tests/, notebooks/, docs/, examples/, scripts/.

3.2 FedLens

Fed communication intelligence engine linking policy language to rates, equities, and volatility moves.

Business case

MDs and strategy teams care about policy language, but reading every document manually is slow. FedLens turns speeches, statements, minutes, and press conferences into a searchable, evaluated copilot tied to market reaction data.

Field	Spec
Primary users	Rates strategists, macro PMs, internal AI leaders, economists, and senior stakeholders who want a high-signal research copilot.
Hard-skill signals	NLP, LLM systems, event studies, embeddings, retrieval, agentic workflows, evaluation design, and human-in-the-loop review.
Stack	Python, SQL, TypeScript, Java, PyTorch, Hugging Face, open-source orchestration, PostgreSQL, FastAPI, React, Plotly, optional Rust for evaluation tooling.
Low-cost deploy	Low-cost path: pgvector or FAISS local store, static React front end, FastAPI and model service in a small container. Azure-first branding is fine, but the repo must run locally.
Launch shape	MVP: Searchable document set, event-study engine, semantic-shift dashboard, and a finance research copilot that drafts policy briefings. Stretch: Multi-document comparison mode, speaker-style fingerprinting, and continuous evaluation dashboards for the agent system.

Architecture

- Document pipeline: ingest FOMC statements, minutes, speeches, and press conference transcripts; clean metadata and chunk by semantic section.
- Model layer: generate sentiment, hawkish/dovish, uncertainty, topic, and embedding-based drift features.
- Market layer: merge communication timestamps with event windows in yields, futures proxies, SPY, and volatility proxies.
- Copilot layer: retrieval plus structured tools for similar speeches, event study, semantic shift, and briefing note generation.

Interfaces

- CLI: fedlens ingest, fedlens embed, fedlens event-study, fedlens brief.
- API endpoints: /documents, /search, /event-study, /briefing, /eval, /agent/run.
- Review queue: a simple HITL table with approval status, comments, and accepted briefing outputs.

Modeling and mathematics

- Event study notation should be visible: abnormal move $AR_t = R_t - E[R_t]$ across pre-defined windows around the document timestamp.
- For semantic drift, cosine similarity and clustering are enough for the main story; deeper topic models can sit in an appendix.

AI and workflow design

- Document ingestion agent, policy-tone agent, market-reaction study agent, explanation/briefing agent, review agent, and human approval checkpoint.
- All agent outputs should be structured: confidence, cited text chunks, cited market windows, and recommended analyst follow-up.

Testing and governance

- Prompt/evaluation harness using curated benchmark questions.
- Document parsing tests and metadata integrity tests.
- Event-study unit tests against known windows.
- Agent trace tests to ensure required citations and approval fields are present.

Suggested repo skeleton

frontend/, api/, src/fedlens_ingest/, src/fedlens_models/, src/fedlens_eventstudy/, src/fedlens_agents/, evals/, tests/, notebooks/, docs/.

3.3 BalanceLab

ALM, NII, EVE, liquidity-gap, and scenario analytics platform with an internal risk copilot.

Business case

This is the most directly bank-usable repo. A treasury or balance-sheet risk team needs a clear way to upload positions, map product behavior, run rate shocks, and explain what the scenario means for NII, EVE, and funding risk.

Field	Spec
Primary users	ALM leadership, treasury, finance transformation teams, internal AI/strategy, and stakeholders who care about earnings sensitivity and scenario explainability.
Hard-skill signals	Institutional risk modeling, scenario analytics, internal tool design, software testing, explainability, ETL realism, and finance-domain communication.
Stack	Python, SQL, TypeScript, Java, C++, FastAPI, React, Plotly, Streamlit, PostgreSQL, DuckDB, Perl/bash for legacy-style file ingestion, optional Azure deployment.
Low-cost deploy	Low-cost path: demo data local-first, React frontend static-hosted, FastAPI in a small container, PostgreSQL optional, DuckDB default for analytics. Azure Container Apps can be shown as the enterprise-flavored path.
Launch shape	MVP: Upload a stylized balance sheet, run a standard scenario set, display NII/EVE/liquidity outputs, and generate a reviewed narrative summary. Stretch: Dynamic balance behavior, deposit beta estimation, hedge overlays, FTP curve management, and approval workflows.

Architecture

- Input layer: uploaded sample balance-sheet files and product mapping rules produce standardized cash-flow and repricing records.
- Analytics layer: gap tables, duration approximations, NII/EVE sensitivity, FTP assignment, and custom scenario engine.
- Application layer: executive dashboard, analyst drilldown, and copilot panel for natural-language scenario exploration.
- Governance layer: scenario catalog, saved assumptions, approval state, and exportable analysis packs.

Interfaces

- CLI: balancelab load, balancelab map-products, balancelab shock, balancelab export-pack.
- API endpoints: /positions, /scenarios, /nii, /eve, /liquidity-gap, /copilot, /assumptions.

- Scenario objects should be explicit JSON with parallel shocks, curve twists, deposit beta assumptions, and balance migration assumptions.

Modeling and mathematics

- Make the core balance-sheet formulas visible. Example: $\Delta \text{NII} \approx \sum_i (\text{RSA}_i - \text{RSL}_i) * \Delta r_i$ over the horizon, with clearer bucketed versions in the UI.
- For valuation sensitivity, show the intuition behind $\Delta \text{EVE} \approx -D_A * A * \Delta y + D_L * L * \Delta y$.
- Duration and repricing are approximate by design; the README should clearly state what is stylized versus what would require production model detail.

AI and workflow design

- Scenario-construction agent, mapping/validation agent, risk-narrative agent, control-checking agent, and human reviewer gate.
- The copilot should answer what changed, what is driving NII/EVE, where the largest maturity/repricing concentrations sit, and which assumptions matter most.

Testing and governance

- Golden-scenario regression tests for NII/EVE outputs.
- Parser tests for uploaded files and mapping rules.
- API tests for scenario creation and calculation outputs.
- Copilot tests to ensure every narrative references scenario inputs and top drivers.

Suggested repo skeleton

frontend/, api/, src/balancelab_core/, src/balancelab_scenarios/, src/balancelab_copilot/, adapters/, tests/, notebooks/, docs/, sample_data/, scripts/.

3.4 VolLab

Derivatives pricing, Greeks, volatility-surface, and stochastic-simulation lab.

Business case

This is the math flagship. It proves the portfolio is not only about dashboards and AI: it shows that the builder can implement pricing logic, explain stochastic assumptions, and benchmark numerical methods.

Field	Spec
Primary users	Quant managers, trading-adjacent leaders, technically sophisticated reviewers, and anyone scanning for hardcore modeling credibility.
Hard-skill signals	Derivatives math, stochastic calculus, numerical methods, performance engineering, testing/benchmarking, and explanatory quant writing.
Stack	Python, Julia, C++, Rust, TypeScript, FastAPI, React, Plotly, optional AWS deployment. Python orchestrates; Julia and C++ provide math/performance depth.
Low-cost deploy	Low-cost path: local or containerized demo with static front end, no need for expensive managed services. Publish benchmark artifacts and screenshots rather than over-engineering cloud spend.
Launch shape	MVP: Vanilla option pricing, Greeks dashboard, stress/PnL explorer, and side-by-side method comparison. Stretch: Rates/FX extensions, local-vol appendix, calibration studies, and a small quantum-inspired optimization appendix.

Architecture

- Core library: pricing functions, Greeks, surface utilities, scenario/PnL routines, and benchmark harnesses.
- Research layer: Julia notebooks/modules for derivations, simulation experiments, and calibration studies.
- Performance layer: C++ kernels for pricing loops, optional Rust CLI for benchmark and scenario runs.
- Presentation layer: React dashboard for option chains, vol surfaces, stress/PnL, and comparison of pricing methods.

Interfaces

- CLI: vollab price, vollab greeks, vollab surface, vollab stress, vollab benchmark.
- API endpoints: /price, /greeks, /surface, /scenario-pnl, /benchmarks.
- Benchmark reports should compare Python, Julia, and C++ implementations where meaningful.

Modeling and mathematics

- Show the stochastic process directly: $dS_t = \mu S_t dt + \sigma S_t dW_t$ for the baseline diffusion case.
- Show a canonical pricing formula such as $C = S N(d1) - K e^{-rT} N(d2)$ and explain when the assumptions fail.
- Greeks, Monte Carlo confidence intervals, local interpolation choices for vol surfaces, and calibration diagnostics should all be visible in docs and notebooks.

AI and workflow design

- VolLab is not primarily an agent project; keep AI light. A derivation explainer or code-to-math assistant is enough.
- The main product is rigor, not chat. Avoid burying the quant story under agent theatrics.

Testing and governance

- Analytical-versus-numerical checks where closed forms exist.
- Greeks finite-difference sanity tests.
- Benchmark tests for performance claims.
- Vol-surface interpolation tests and Monte Carlo reproducibility checks.

Suggested repo skeleton

frontend/, api/, python/, julia/, cpp/, rust_cli/, tests/, benchmarks/, notebooks/, docs/, examples/.

3.5 RegimeOS

Regime detection and decision-support orchestration engine for macro, policy, and market-risk synthesis.

Business case

Senior decision-makers often need one answer to a harder question: what regime are we in, what changed, and what should we focus on? RegimeOS combines macro data, policy language, and market behavior into an explainable state engine with portfolio and risk guidance.

Field	Spec
Primary users	Strategy teams, multi-asset thinkers, enterprise AI groups, and leaders who want synthesis rather than a pile of charts.
Hard-skill signals	Bayesian/probabilistic reasoning, multi-agent orchestration, state estimation, portfolio/risk translation, systems thinking, and stakeholder communication.
Stack	Python, SQL, TypeScript, Go, Julia, FastAPI, React, Plotly, DuckDB/PostgreSQL, probabilistic modeling tools, optional Java integration.
Low-cost deploy	Low-cost path: local-first signals store, static front end, small API container, scheduled jobs via GitHub Actions or a tiny container host, optional Azure or AWS branding layer.

Launch shape	MVP: Rule-based or probabilistic regime engine, explainability dashboard, and a multi-agent briefing that summarizes the current state and decision priorities. Stretch: Walk-forward evaluation, richer transition modeling, portfolio/risk templates by regime, and stronger cross-project integration.
--------------	---

Architecture

- Signal layer: pull selected features from Atlas, text/policy signals from FedLens, and market/volatility indicators from a curated market data set.
- State layer: regime definitions can be rule-based initially, then upgraded to probabilistic state assignment with uncertainty outputs.
- Decision layer: convert regime states into watchlists, allocation/risk notes, and scenario priorities.
- Agent layer: macro-state agent, policy-signal agent, market-risk agent, recommendation agent, explanation agent, plus human checkpoint.

Interfaces

- CLI: regimeos refresh, regimeos score-state, regimeos explain, regimeos publish-note.
- API endpoints: /signals, /state, /uncertainty, /recommendations, /agent/run, /approval-queue.
- Go service can own orchestration jobs and scheduled refresh tasks if you want a clear systems cameo.

Modeling and mathematics

- State probability should be explicit. Even a simple notation like $p(z_t = k \mid x_t)$ communicates that the system is assigning regimes conditionally on observed signals.
- Uncertainty, confidence, and transition logic matter more than squeezing maximum complexity into version one.
- A Bayesian or hidden-state appendix can sit beside a simpler operational classifier for the main app.

AI and workflow design

- This is one of the strongest enterprise-AI showcases in the suite. Agents should gather evidence, reconcile disagreement, and surface a recommendation with confidence and citations.
- Human in the loop is essential: recommendations are draft decision support, not auto-trades.

Testing and governance

- State-classification regression tests.
- Integration tests that prove Atlas and FedLens outputs can be consumed cleanly.
- Agent evaluation tests for recommendation structure and evidence citation.
- Backtest-lite tests for regime-to-action logic, clearly labeled as research rather than live strategy.

Suggested repo skeleton

frontend/, api/, src/regimeos_signals/, src/regimeos_state/, src/regimeos_agents/, orchestration_go/, tests/, notebooks/, docs/, evals/.

4. Cross-Project Integration

The portfolio becomes more impressive when the projects can talk to one another. The goal is not tight runtime coupling across all demos, but shared contracts and believable information flow.

From	To	Artifact passed	Why it matters
Atlas	RegimeOS	Macro feature blocks and regime snapshots	Shows a shared macro backbone instead of duplicate data logic
FedLens	RegimeOS	Policy-language signals and event-study outputs	Shows how policy information becomes a structured state input
BalanceLab	Landing repo	Screenshots, scenario notes, case-study writeups	Shows practical internal-tool readiness
Vollab	Landing repo	Benchmarks, derivation notes, demo GIFs	Shows hard quant depth without forcing runtime coupling
All repos	Master repo	Shared badges, design system, issue templates, docs links	Makes the portfolio feel like one serious suite

Shared README structure

- What problem the project solves and why a finance team should care.
- What data and tools it uses.
- What the system architecture looks like.
- What mathematics are important enough to explain explicitly.
- How to run it locally at low cost.
- How the project should evolve if it were taken from portfolio demo to production pilot.

Shared definition of done

- A clean README, a working local demo path, sample data or fixtures, screenshots/GIFs, and a test badge.
- At least one exportable artifact per repo: PDF report, notebook, benchmark table, or scenario memo.
- A product-grade UI for the core user path and a research-grade surface for technical review.
- A short business pitch section aimed at MDs or strategy leaders and a technical architecture section aimed at builders.
- Visible limitations and future work so the work feels honest rather than overclaimed.

Final positioning note

Taken together, these five projects position you as an enterprise AI and financial analytics builder who can move between macro data systems, policy-language intelligence, balance-sheet risk, derivatives math, and decision-support orchestration.